Flujo de la práctica: experimento SDD

Esta práctica fue realizada sobre un trabajo práctico de la materia Taller de Programación, en el que se realizó un juego arcade de carreras en 2D, basado en una arquitectura cliente-servidor y con partidas multijugador. Para este experimento, se planteó agregar a la pista Power-Ups que otorguen al jugador un efecto de velocidad, regeneración de vida o de inmunidad ante choques.

Tipo	Efecto	Duración
Nitro	Velocidad máxima y aceleración × `nitro_multiplier` (sobre los valores con upgrades)	`nitro_duration`
Reparación	+ `repair_amount` de salud, sin superar la salud máxima	Instantáneo
Escudo	Ignora el daño por choques	`shield_duration`

El primer paso realizado previo a comenzar el flujo SDD, dado que la práctica era sobre un proyecto brownfield, fue la creación de un archivo de especificación, `TODO.md`, con la información necesaria de la feature que se quería agregar. En esta especificación se detallaron los requerimientos iniciales de los hitos y se agregaron pruebas de aceptación como mecanismo de feedback para el modelo. También se configuraron hooks en el proyecto:

- `pre-commit-test`: bloquea un git commit si los tests no están todos en verde.
- `pre-commit-checks`: bloquea un git commit a menos que pasen clang-format, cpplint y cppcheck.
- `modified-file-checks`: luego de modificar un archivo C++, se le da formato y se corre cppcheck.

El flujo elegido para la implementación de la feature estuvo compuesto únicamente por comandos de OpenSpec, con una sesión de Claude Code (Opus 5.5) para cada hito.

Primero se realizó un único `/opsx:explore`, que generó la especificación general del proyecto y los primeros hitos. Dentro de cada hito se siguió el ciclo: `/opsx:propose <Nombre del hito>` -> `/opsx:apply` -> `/opsx:verify` -> `/opsx:sync` -> `/opsx:archive`

En el plan original, la feature de Power-Ups se dividió en cuatro hitos.

#	Hito
1	Configuración y modelo de efectos en el servidor (sin física ni red)
2	Power-ups en el mundo físico (sensores Box2D, spawn, reaparición)
3	Game loop y protocolo (DTOs, snapshots)
4	Render y sonido en el cliente (sprites, HUD, sonido)

1. Explore inicial

El objetivo de este paso fue definir la constitución de un proyecto brownfield a partir de la documentación y el código existentes. Antes de generar archivos, el modelo debe entender el proyecto y entrevistar al usuario para completar la información que falte. La misión y el stack no cambian durante el flujo de trabajo; el

roadmap sí se actualiza con el estado de cada hito y, como se ve en la sección 3, se extendió con dos hitos nuevos. OpenSpec no tiene un comando `constitution`, así que decidí recrearlo para tener archivos con algo más de contexto: `mission.md`, `tech-stack.md` y `roadmap.md`. Estos archivos no cambian nada de la configuración original de OpenSpec; únicamente se referencian desde `openspec/config.yaml`, el archivo que leen las skills de OpenSpec.

Luego de correr `openspec init` en la raíz del proyecto, el primer comando que se realizó fue el explore:

```
/opsx:explore
En este proyecto brownfield, analiza el README.md y el TODO.md. Explora
también la estructura del repositorio para entender cómo está organizado y
qué tecnologías utiliza.

Antes de escribir o modificar archivos, entrevístame usando la herramienta
AskUserQuestion. Pregúntame sobre:
- La misión del proyecto y el problema que busca resolver.
- La audiencia objetivo y sus necesidades principales.
- Las brechas, dudas o decisiones pendientes sobre el stack tecnológico.

Cuando haya respondido, revisa la información disponible y crea o actualiza
estos archivos:
- openspec/constitution/mission.md
- openspec/constitution/tech-stack.md
- openspec/constitution/roadmap.md

Usa la documentación y mis respuestas como fuentes. Si falta información
importante, indícala como pendiente en lugar de asumirla. Al terminar,
resume brevemente los archivos creados o modificados y las decisiones que
quedaron abiertas.
```

Este prompt llevó a la creación de los archivos allí descriptos y, a su vez, del archivo de configuración de OpenSpec, que es el que leen las skills de la herramienta.

2. Ciclo de trabajo SDD

2.1 Propose

El ciclo de cada hito en OpenSpec comenzó con `/opsx:propose <Nombre del hito>`. Con esta skill se crearon 4 archivos de especificación para el hito:

- `proposal.md`: explica el porqué (*Why*), qué cambia, qué queda fuera del alcance del hito, etc.
- `design.md`: decisiones principales de diseño del código.
- `tasks.md`: el hito dividido en las tareas que hay que completar para terminarlo.
- `spec.md`: escenarios y requerimientos.

Las reglas de `openspec/config.yaml` obligan a que el proposal indique qué hito del roadmap implementa y qué queda fuera de alcance, a que cada scenario Given/When/Then nombre el test de

GoogleTest que lo verifica (`Suite.Test` y su archivo en `tests/`) y a que las primeras tareas escriban los tests de aceptación y comprueben que fallan.

2.2 Apply

`/opsx:apply` implementa el change: lee los artefactos del hito y recorre `tasks.md` en orden, marcando cada tarea como hecha a medida que la completa.

2.3 Update

`/opsx:update <cambio>` revisa los artefactos de planificación ya creados (proposal, design, specs y tasks) y los mantiene coherentes entre sí. No toca código: si después hace falta implementar algo, se vuelve a correr `/opsx:apply` . No fue un paso fijo del ciclo; se usó cuando hizo falta corregir el plan:

- Hito 3: antes del apply, para indicar que las coordenadas de edificios y decoraciones de los mapas estaban en `server/world/object_rendering/decorations_*.json` , y para revisar otras decisiones del design.
- Hito 3: después del verify, para dejar asentado en los artefactos el arreglo de un warning.
- Hito 4: antes del apply, para fijar el tamaño del sprite (cuadrado, del ancho del auto compact y con fondo transparente).

2.4 Verify

`/opsx:verify` comprueba, antes de archivar, que la implementación coincida con los artefactos. El resultado es un reporte con hallazgos `CRITICAL` , `WARNING` y `SUGGESTION` . Según la constitution, un hito no está terminado mientras el verify tenga un `CRITICAL` (gate entre hitos). En la práctica, después de cada verify se le pidió al modelo que arreglara los warnings y aplicara las sugerencias, y que marcara las tareas que solo podían comprobarse a mano (por ejemplo, la prueba manual de una partida) una vez probadas. Después se volvía a verificar.

2.5 Sync

Cada change no reescribe las specs principales: trae *delta specs* en `specs/<capacidad>/spec.md` con secciones `ADDED` , `MODIFIED` , `REMOVED` y `RENAMED Requirements` . `/opsx:sync` aplica esos deltas a las specs principales en `openspec/specs/` , haciendo un merge inteligente: agrega o modifica un scenario sin copiar el requirement entero. Así, las specs principales describen siempre cómo se comporta el sistema hoy, y cada hito parte de ellas.

2.6 Archive

`/opsx:archive` cierra el change: comprueba que los artefactos y las tareas estén completos, ofrece hacer el sync si las delta specs todavía no se aplicaron y mueve la carpeta del change a `openspec/changes/archive/AAAA-MM-DD-<nombre>/` . Ahí queda como registro histórico del hito. Además, si cambia el protocolo o algo visible para el usuario, se actualizan el `README.md` y los manuales.

2.7 Fin de sesión

En esta práctica, para comparar el trabajo que llevó cada hito, se creó una sesión nueva de Claude Code por hito, y así se pudo comparar el uso del modelo entre ellas. Además del tiempo de uso, el comando `/usage`

brinda otros parámetros interesantes de cada sesión.

# Hito	Total Cost (usd)	Total duration (API)	Total duration (wall)	Tokens input	Tokens output	Tokens cache
1	$5.44	13m 10s	1h 2m 14s	1.9k	67.4k	12.9m
2	$5.10	9m 26s	26m 11s	2.2k	68.1k	11.8m
3	$13.28	27m 18s	1h 10m 40s	4.5k	133.9k	40.0m
4	$9.98	16m 15s	54m 31s	3.4k	104.6k	27.7m

- Total duration (API): tiempo que el agente estuvo trabajando.
- Total duration (Wall): tiempo total de la sesión.

3. Cambios de alcance

En las pruebas manuales de los hitos 3 y 4 surgieron dos bugs que no habían sido contemplados por la especificación inicial:

- Algunos Power-Ups aparecían fuera de la calle, en lugares a los que el auto no puede acceder.
- Los Power-Ups no estaban configurados con las capas que usa el juego para estar encima o debajo de un puente, por lo que un auto podía pasar por debajo y agarrar un Power-Up que estaba encima.

Para solucionar estos dos problemas se realizó un segundo `/opsx:explore`, que, luego de responder algunas preguntas del agente, llevó a la creación de dos hitos nuevos:

```
/opsx:explore
tengo dos cambios que proponer para un nuevo hito que surgio en medio de la implementacion: power-ups-spawn-sobre-calle (ver task archivado del render de power ups) y luego hay algunos power ups que se superponen con los puentes, un power up que deberia estar en el puente y se ubica justo en el borde queda la mitad del pówer up abajo y la otra mitad siguen en la calle que no pertenece al puente. Ademas deberia ser por capas, un power up que esta en un puente no deberia poder ser agarrado por un auto que pasa por debajo.
```

Este nuevo explore agregó dos hitos nuevos al roadmap de la feature:

#	Hito
5	Power-ups sobre la calle (grilla de calles y camino por capas entre checkpoints)
6	Power-ups por capas en los puentes (recolección por capa, borde del puente, render)

Al terminar los cuatro hitos iniciales, se realizó el ciclo de trabajo SDD para estos dos y, al término de cada sesión, se recolectó la información de uso con `/usage`:

# Hito	Total Cost (usd)	Total duration (API)	Total duration (wall)	Tokens input	Tokens output	Tokens cache
5	$11.81	27m 35s	59m 4s	1.5k	139.9k	32.8.9m
6	$7.15	16m 55s	42m 31s	1.9k	87.3k	17.4m

4. Situaciones destacadas

A lo largo de los hitos hubo algunas situaciones en las que la intervención humana cambió el resultado del flujo: revisar lo que el modelo planificaba, probar el juego a mano y corregir criterios de aceptación. Las detallo a continuación con los prompts que tuve que mandar en cada caso.

4.1 Revisión del design antes del apply (Hito 3)

Al terminar el `/opsx:propose` del hito 3, el modelo dejó una pregunta abierta en el `design.md`: el test 3.7 (el escudo protege de un choque y, al expirar, el choque hace daño) necesitaba que el auto chocara contra algo en el recorrido de la carrera de test, y no sabía contra qué. Para resolverlo le indiqué dónde creía que estaban las coordenadas de los edificios:

```
/opsx:update en la ruta /server/world/object_rendering estan los archivos decorations_*.json que contienen las coordenadas de los edificions y demas de los mapas del juego
```

El modelo revisó esos archivos y encontró que mi dato no era del todo correcto: las decoraciones solo se usan para dibujar en el cliente, y las paredes con las que choca el auto se cargan desde `server/world/collidables/on_ground_collidables_liberty_city.json`. Con eso ubicó la pared real del recorrido, definió la maniobra del test y agregó un test de control sin escudo, porque un choque lento puede hacer 0 de daño y el 3.7 pasaría sin probar nada.

4.2 La prueba manual reveló requisitos que la spec no tenía (Hitos 4 → 5 y 6)

Al probar a mano el hito 4 noté que algunos power-ups aparecían sobre edificios o paredes. Lo informé con un update, junto con un problema del indicador de duración:

```
/opsx:update las carreras funcionan bien. Dos puntos acerca de los power ups. Deberian aparecer entre dos checkpoints pero siempre sobre la calle, no sobre un edificio o pared porque se pierde un poco la inmersion. [...]
```

El modelo encontró las máscaras de calles que usa el editor (`editor/EditorMapas/imagenes/00X - B_STREETS - <ciudad>.png`) y midió el problema en las carreras existentes: 23 de las 108 posiciones de power-ups caían fuera de la calle, casi siempre porque el tramo dobla una esquina y el punto medio en línea

recta queda dentro de la manzana. Como el cambio excedía al hito 4, lo dejamos anotado en el `tasks.md` como un change aparte y el hito 4 se cerró solo con la corrección del indicador.

Con esto armé el `/opsx:explore` de la sección 3.

En el explore el modelo midió que 8 posiciones caían enteras dentro de un puente y 7 cruzaban su borde, y encontró algo que ninguno de los dos había previsto: debajo de los puentes la máscara es toda calle, así que el camino del hito 5 podía "subir" al puente desde la calle de abajo. Propuso que el camino del hito 5 ya recorra las capas igual que los autos, y que el hito 6 use la capa de cada punto del camino para decidir quién agarra cada power-up. Validé el criterio, incluido el caso de los puentes sobre el agua. Así se agregaron los hitos 5 y 6 al roadmap, con su Background y sus criterios. Los tests automáticos de los hitos 1 a 4 pasaban; el problema solo apareció jugando, porque la spec original nunca dijo que el punto medio tuviera que estar sobre la calle.

4.3 Corrección de un criterio de aceptación frágil (Hito 5)

Al revisar los criterios nuevos que agregó el explore, vi que el 5.11 comprobaba que todos los power-ups de las carreras reales de `server/races/` quedaran sobre la calle, y que además fijaba la cantidad en 108. Como las carreras se pueden crear desde el editor, ese test dejaría de funcionar en cuanto alguien agregara una:

```
correccion en los test del 5.11, en el editor se pueden agregar carreras, por lo que si en algun momento el cliente ccrea una carrera este test dejaria de funcionar
```

El modelo cambió el 5.11 (y el 6.15, que tenía el mismo problema) para que use una copia fija de las carreras en `tests/data/races/`, sin depender de la cantidad. Para cubrir las carreras nuevas agregó el criterio 5.12: una carrera creada desde el editor no puede hacer fallar la carga aunque tenga tramos sin camino, y en ese caso el power-up vuelve al punto medio entre los checkpoints. La corrección se hizo antes del `/opsx:propose` del hito 5, así que la spec ya nació con la versión corregida.

5. Análisis con SonarQube Cloud

Al terminar los seis hitos, quise medir si el flujo SDD había afectado la calidad del código. Para eso analicé el proyecto con SonarCloud, siempre con la misma configuración (`sonar-project.properties`, la misma base de compilación y la misma cobertura de gcovr), así los resultados se pueden comparar:

- **Pre-SDD**: commit antes de comenzar con SDD.
- **Post-SDD (con el arreglo de issues)**: al finalizar los 6 hitos.

5.1 Pre-SDD contra post-SDD

Indicador	Pre-SDD	Post-SDD
Quality Gate	No calculado	Passed

Indicador	Pre-SDD	Post-SDD
Issues abiertas	23	0
Confiabilidad	E	A
Seguridad	A	A
Mantenibilidad (deuda)	A (53 min)	A
Cobertura	89,3 %	90,1 %
Cobertura en código nuevo	–	86,2 %
Líneas de código	9.392	10.637
Complejidad ciclomática	1.554	1.848
Complejidad cognitiva	1.033	1.250

Los hitos agregaron unas 1.200 líneas de código sin sumar ninguna issue, y la cobertura subió casi un punto, consecuencia de que cada scenario tuviera su test. La complejidad crece junto con el código: por línea pasa de 0,165 a 0,174.

5.2 Complejidad por archivo

Para ver si los cambios del flujo SDD influyeron mucho en la complejidad, comparé por archivo los dos análisis. En total el proyecto sumó 217 de complejidad cognitiva (1.033 → 1.250), 294 de ciclomática (1.554 → 1.848) y 1.245 líneas de código. Estos son los archivos que más complejidad sumaron:

Archivo	Cognitiva pre → post	Ciclomática pre → post	Métodos pre → post
`server/streets_grid.cpp` (hito 5)	— → 115	— → 133	— → 29
`server/tools/streets/streets_grid_generator.cpp` (hito 5)	— → 15	— → 15	— → 3
`server/power_up.cpp` (hitos 2 a 6)	— → 14	— → 37	— → 23
`client/power_up_view.cpp` (hito 4)	— → 12	— → 16	— → 5
`server/game_loop.cpp` (hitos 3 a 6)	77 → 86	111 → 120	43 → 46
`client/game_logic.cpp` (hito 4)	113 → 120	81 → 90	15 → 17
Proyecto completo	**1.033 → 1.250**	**1.554 → 1.848**	**754 → 854**

Para comparar archivos de distinto tamaño, calculé las dos complejidades por línea y por método:

Archivo	Cognitiva x método pre → post	Ciclomática x método pre → post
`server/streets_grid.cpp` (hito 5)	— → 3,97	— → 4,59
`server/tools/streets/streets_grid_generator.cpp` (hito 5)	— → 5,00	— → 5,00
`server/power_up.cpp` (hitos 2 a 6)	— → 0,61	— → 1,61
`client/power_up_view.cpp` (hito 4)	— → 2,40	— → 3,20
`server/game_loop.cpp` (hitos 3 a 6)	1,79 → 1,87	2,58 → 2,61
`client/game_logic.cpp` (hito 4)	7,53 → **7,06**	5,40 → **5,29**
Proyecto completo	**0,110 → 0,118**	**1,37 → 1,46**

- **Escala de SonarQube:** según la herramienta de análisis, ninguno de los archivos tiene un nivel de complejidad ciclomática o cognitiva por método que necesite intervención. Para la complejidad cognitiva, el rango de riesgo bajo es 1-15, y el valor máximo del proyecto es **7,06**. Para la complejidad ciclomática, el rango de riesgo bajo es de 1-10, y el valor máximo del proyecto es **5,29**.
- **El promedio del proyecto sube poco:** por línea, la complejidad cognitiva pasa de 0,110 a 0,118 y la ciclomática de 0,165 a 0,174; por método, de 1,37 a 1,46 y de 2,06 a 2,16.

6. Analisis con CodeScene

CodeScene apunta a: **qué tan fácil es entender y modificar el código**. Calcula un **Code Health** de 1 a 10 por archivo, a partir de factores que se asocian con mayor costo de mantenimiento y defectos: funciones complejas, condicionales anidados, código duplicado, funciones con muchos argumentos, etc.

Según el puntaje, cada archivo cae en una de tres calificaciones:

- **Sano (verde, 9 a 10):** código fácil de leer y de cambiar.
- **Atención (amarillo, 4 a 8,99):** tiene code smells que conviene revisar.
- **Problemático (rojo, 1 a 3,99):** código caro de mantener y con más riesgo de defectos.

La comparación, igual que con sonar, estuvo dada por:

- **Pre-SDD**: el mismo de SonarCloud, antes de comenzar el flujo SDD.
- **Post-SDD**: al terminar los 6 hitos.

6.1 Métricas generales

Métrica	Pre-SDD	Post-SDD	Qué mide
Code Health del proyecto	9,28	9,14	Promedio del Code Health de los archivos
Hotspot Code	9,29	9,41	Muestra la salud del código que

Métrica	Pre-SDD	Post-SDD	Qué mide
Health			más se modifica.
Peor archivo	6,36 (`client/game_logic.cpp`)	5,72 (`server/streets_grid.cpp`)	El archivo con el Code Health más bajo.
Código verde / amarillo / rojo	75,8 % / 24,2 % / 0 %	73,1 % / 26,9 % / 0 %	Porcentaje de líneas que cae en cada banda.
Archivos con 10,00	63 de 86	69 de 93	Archivos sin ningún code smell
Hallazgos	94	111	Cantidad total de code smells
Funciones complejas	20	25	Funciones con complejidad ciclomática alta (9 o más).

El Code Health bajó de 9,28 a 9,14, y casi todo se explica por un archivo nuevo: `server/streets_grid.cpp`, del hito 5 (5,72). Concentra 16 de los 18 hallazgos nuevos.

De los 94 hallazgos que había antes del SDD, 88 siguen iguales, 5 cambian de valor y solo 1 se resolvió (`GameLoop::run` dejó de ser compleja). Es decir, el flujo SDD no empeoró el código que ya existía, pero tampoco corrigió su deuda. Los hotspots mejoran un poco (9,29 → 9,41). Esto se puede deber que el flujo de SDD solo trabaja en el alcance de las especificaciones, si estos issues no son tratados como parte de un hito el agente no los va a arreglar.

6.2 Code Health por carpeta

Carpeta	Archivos pre → post	Líneas pre → post	Code Health pre → post
`client`	56 → 58	3.994 → 4.189	9,01 → 9,01
`server`	42 → 49	3.183 → 4.140	9,53 → **9,13**
`common`	51 → 55	1.222 → 1.283	9,36 → 9,39
`editor`	19 → 20	832 → 857	9,47 → 9,46

- **Code Health:** promedio de los archivos de la carpeta, ponderado por líneas, para que un archivo grande pese más que uno chico.

La caída se concentra en `server`, que es donde el SDD sumó más código, ya que es la que presenta el archivo mas comeplejo, el mapeo de las calles de cada mapa. Las otras tres carpetas quedan prácticamente iguales. En `client`, `game_logic.cpp` sigue siendo el archivo con más deuda del proyecto, pero

conserva su puntaje aunque el hito 4 le agregó código, igual que en SonarCloud, donde su complejidad por método bajó un poco.

Se debe aclarar que este segundo analisis se realizo luego de terminar los 6 hitos, por lo que estos resultados no se tuvieron en cuenta en el flujo de SDD, si no que fue realizado para familiarizarse con la herramienta de CodeScene.

7. Conclusiones de las herramientas de analsis

Los analisis sirven mucho para observar como trabaja el agente a partir de la metodlogía de SDD. Si en las especificaciones no se indican estos errores encontrados por la herramienta, al estar fuera de alcance, no serán tratados. Las dos herramientas tampoco miden lo mismo. Después del arreglo, SonarCloud queda con 0 issues, mientras que CodeScene mantiene sus 111 hallazgos sin cambios. Que Sonar no marque issues no significa que el código sea fácil de mantener, por eso puede ser conveniente utilizar las dos. Una buena forma de trabajar con estas herramientas sería incluirlas en el flujo para que el agente construya un código mantenible y escalable:

- Hacer un primer análisis antes de comenzar, para conocer la deuda existente y decidir si se trata en un hito propio.
- Agregar criterios de calidad a las especificaciones o a `openspec/config.yaml`, por ejemplo: “0 issues nuevas en SonarCloud” o “ningún archivo nuevo por debajo de 9 de Code Health de CodeScene”. Así se tratarían como un criterio de aceptación más, igual que hoy se exige que cada scenario tenga su test.
- Volver a analizar en el `/opsx:verify` de cada hito, para corregir los problemas que agregue el agente antes de archivarlo.

De esta manera, el código crecería de la mejor manera posible a medida que avanzan los hitos. Como el experimento se hizo sobre una sola feature y los análisis se realizaron al final, estas conclusiones son una observación de este caso y no algo general; se deberia de probar el flujo con las herramientas integradas desde el principio para confirmar que sea el mejor uso.